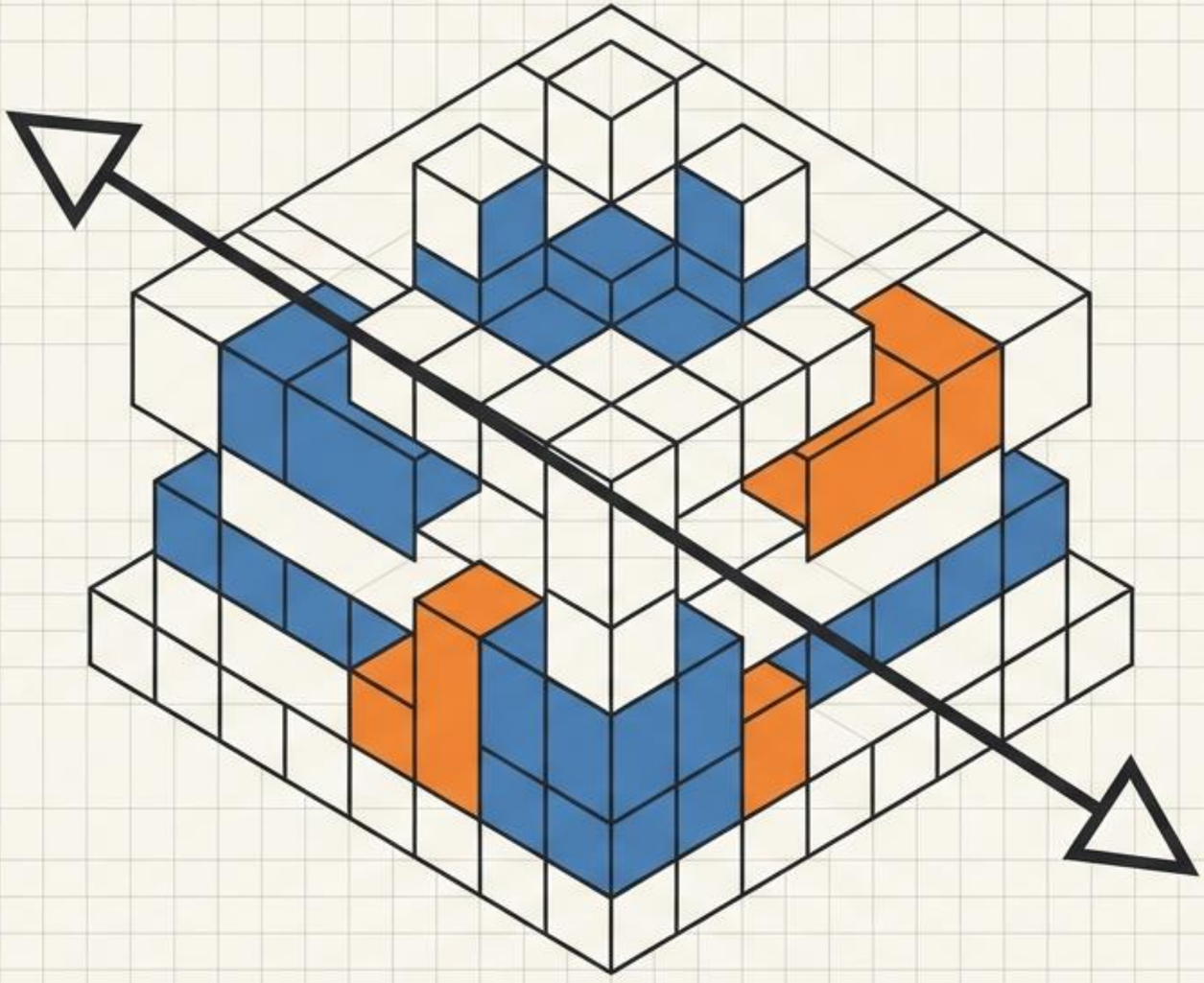


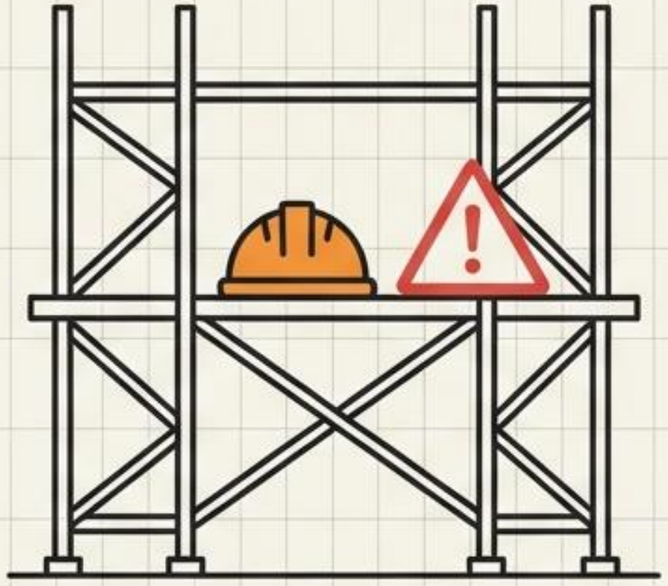
Arquitetura da Memória em C

Desmistificando ponteiros, stack, heap e alocação dinâmica.

NotebookLM

O Poder e a Responsabilidade

Linguagens Gerenciadas

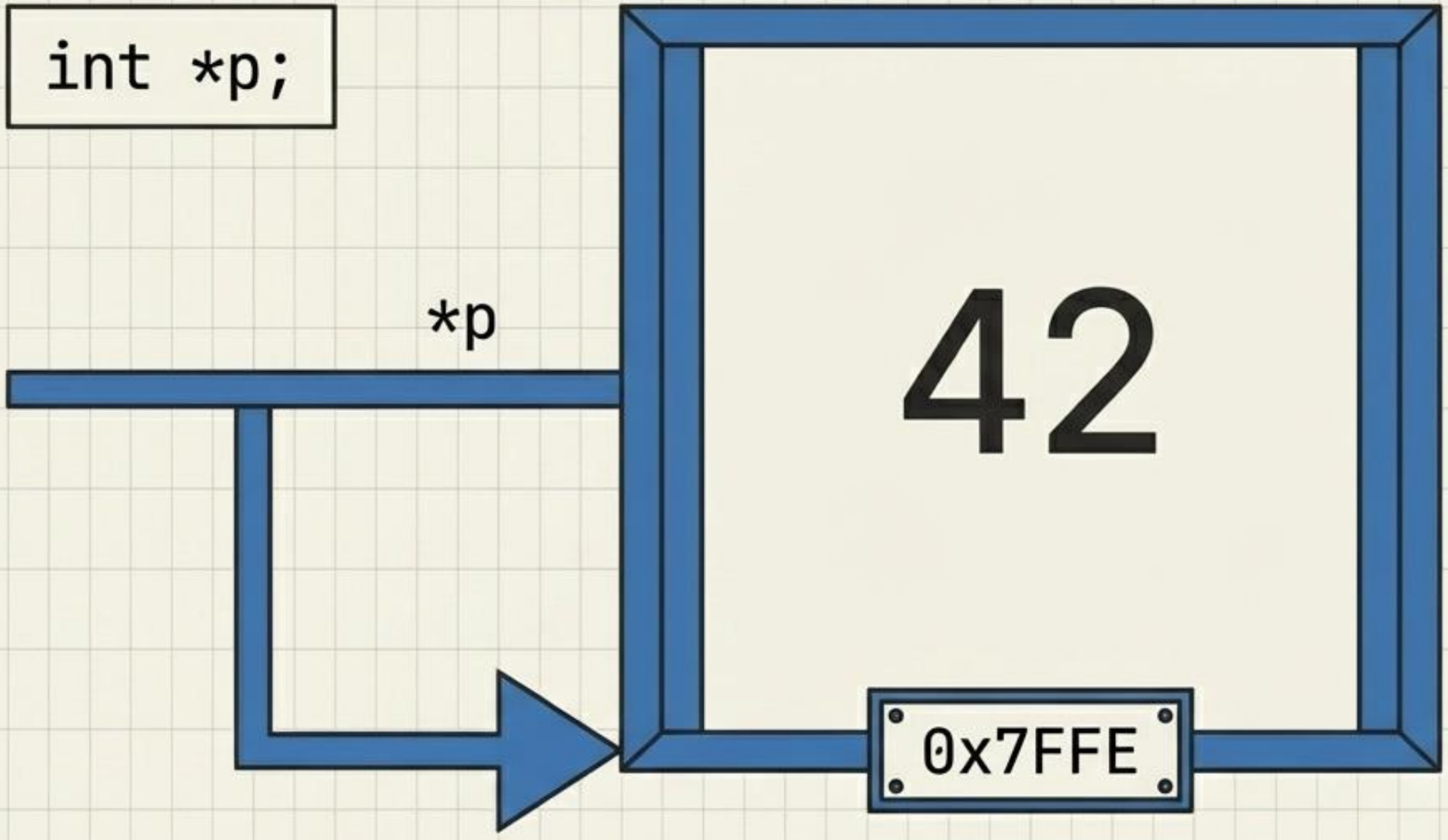


Linguagem C

- Na linguagem C, o gerenciamento de memória é explícito.
- Não existe Garbage Collector.
- O programador deve solicitar memória diretamente ao Sistema Operacional e, obrigatoriamente, devolvê-la quando não for mais necessária.

NotebookLM

A Chave da Memória: O Ponteiro



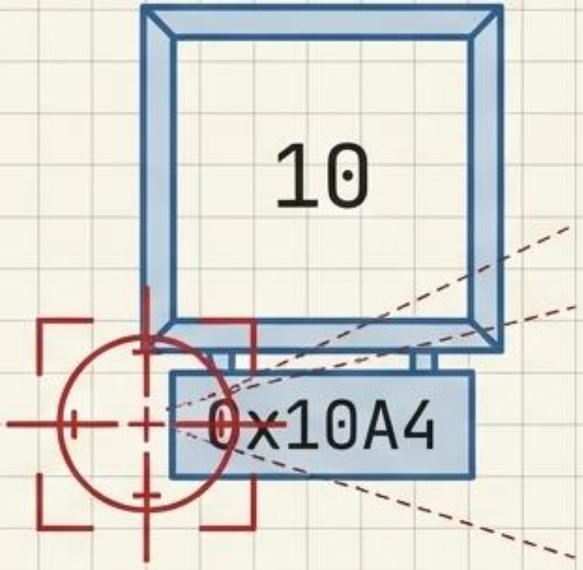
Um ponteiro não armazena dados reais.
Ele é uma variável projetada exclusivamente para armazenar um endereço de memória.
Ele é o mapa, não o tesouro.

NotebookLM

Navegando na Planta Baixa

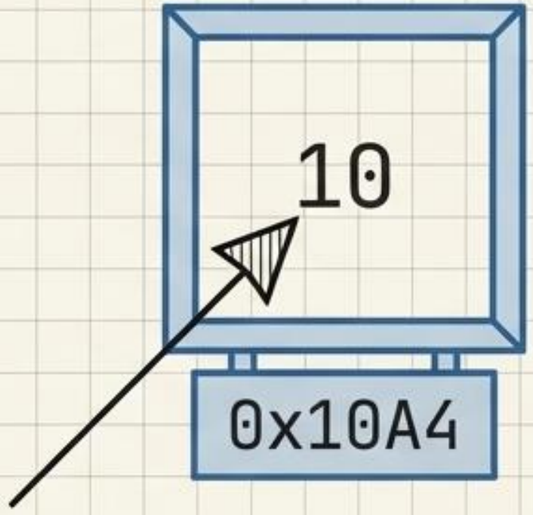
```
int x = 10;
```

Operador &



Obtém o endereço (as coordenadas) de uma variável.

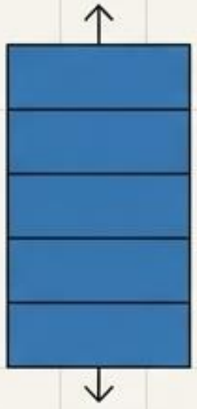
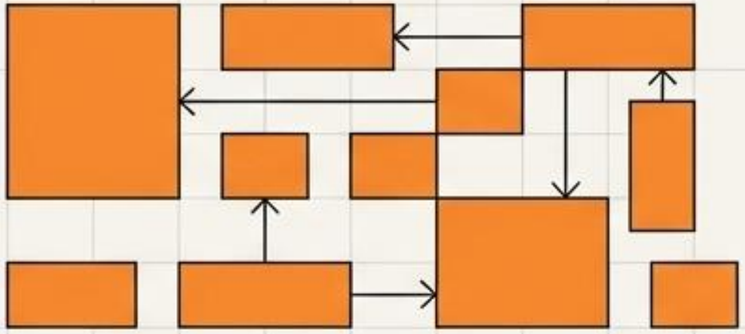
Operador *



Acessa o valor armazenado no endereço (desreferenciação).

NotebookLM

Mapeamento Geográfico: Stack vs. Heap

		
Attribute	**Stack**	**Heap**
Tamanho	Fixo (ex: 8 bytes para ponteiros)	Variável e dinâmico
Conteúdo Típico	Ponteiros locais e variáveis de função	Conteúdo de structs e grandes blocos
Liberação	Automática no fim da função	Manual via programador

NotebookLM

Requisitando Espaço Dinâmico

```
#include <stdlib.h>
```

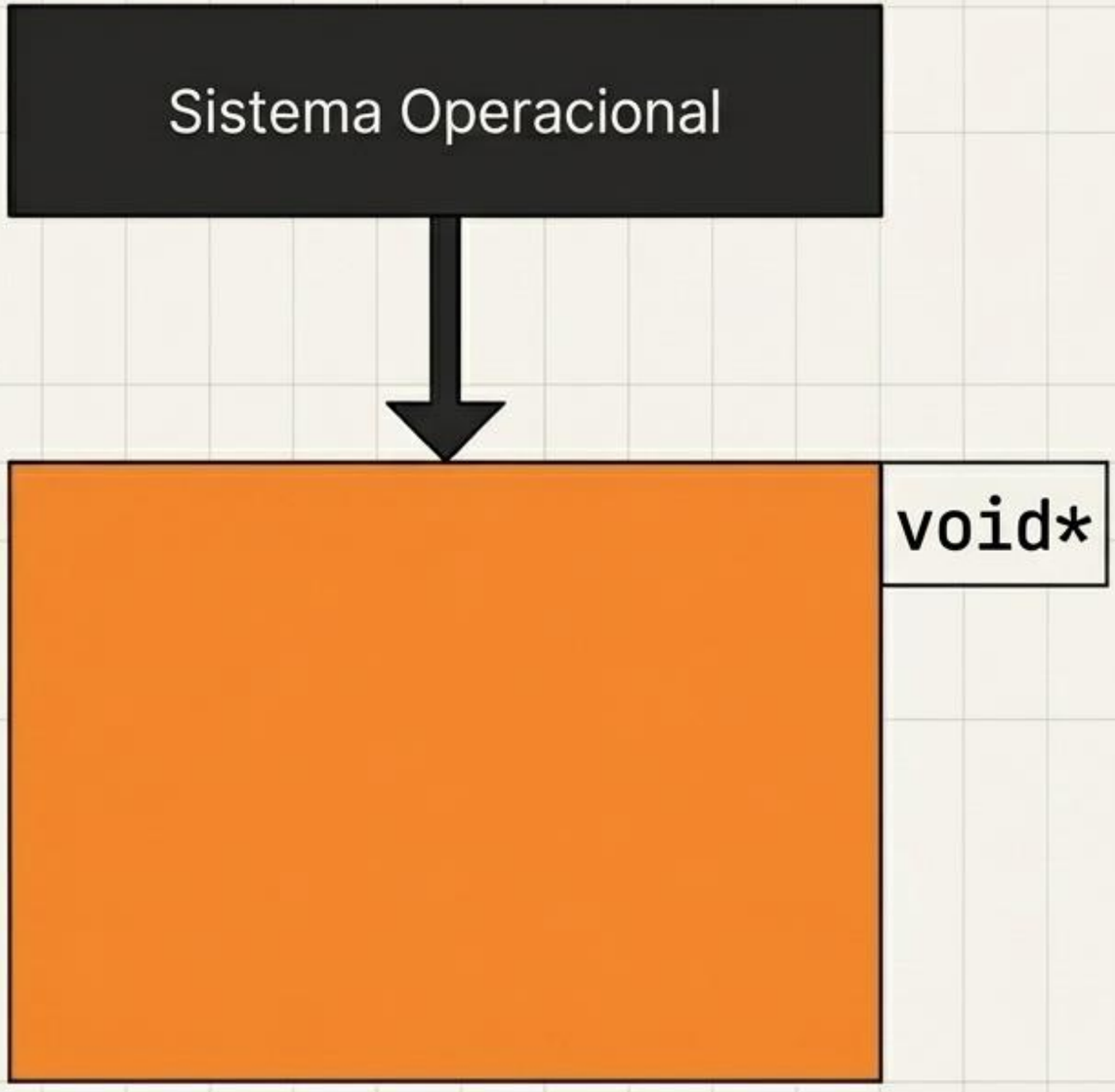
Para criar estruturas que crescem durante a execução, pedimos espaço ao Heap.

`malloc(tamanho)`

Aloca um bloco de bytes no Heap e retorna um ponteiro genérico (void*).

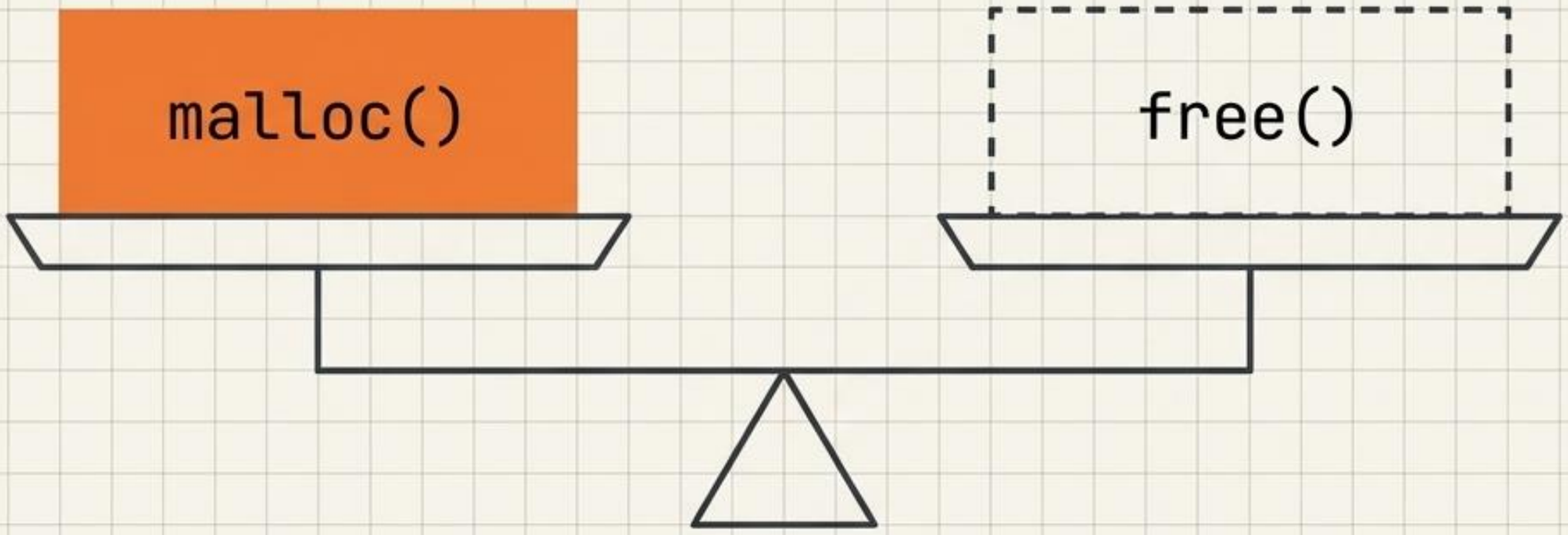
`free(ponteiro)`

Libera o bloco para ser reutilizado.



NotebookLM

O Contrato da Memória



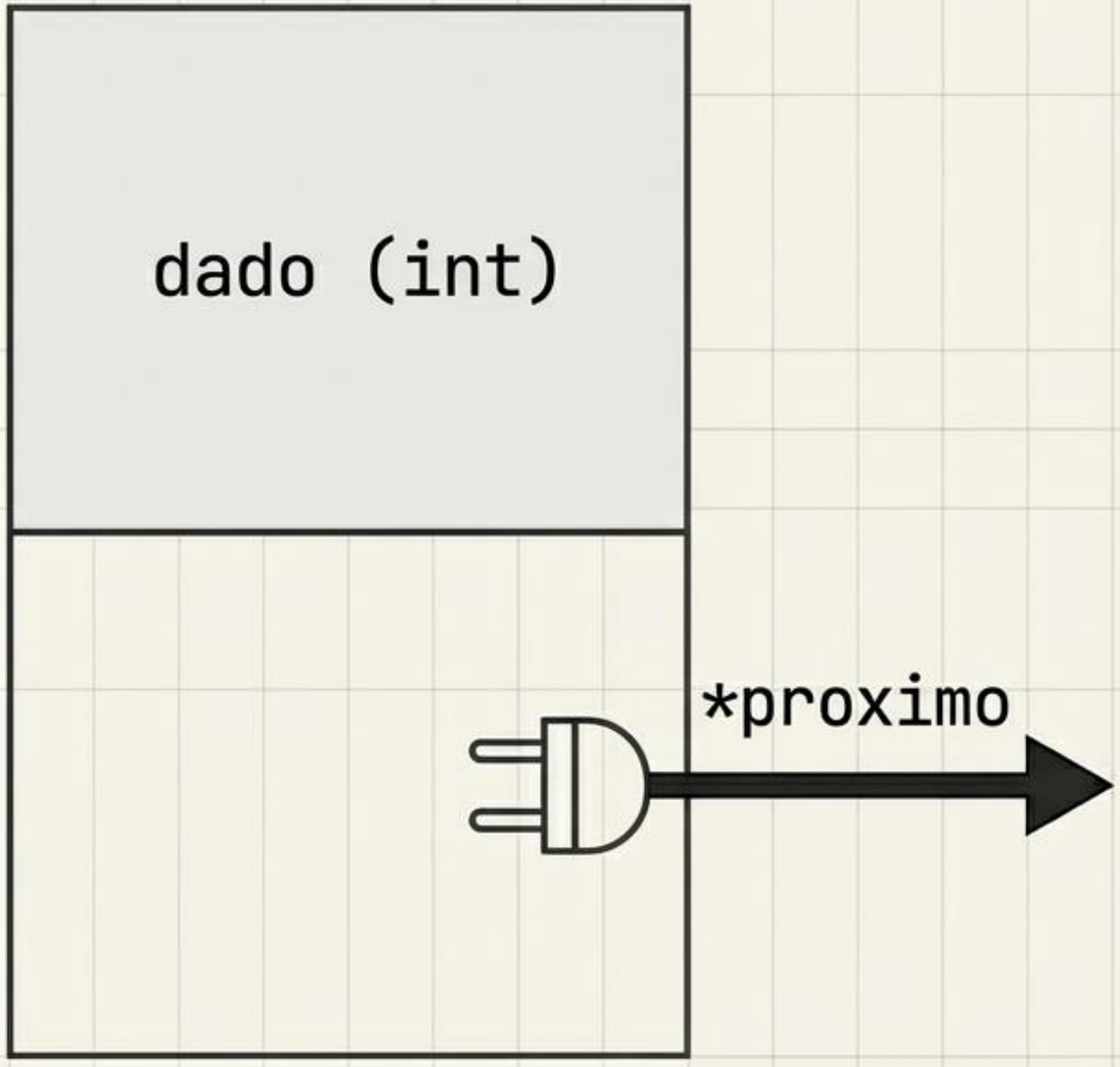
A Regra de Ouro: Para cada malloc executado, deve haver um free correspondente.

NotebookLM

A Planta do Edifício: O Nó (Node)

A unidade básica. Utilizamos struct para definir esta planta.

```
typedef struct No {
    int dado;
    struct No *proximo;
} No;
```



NotebookLM

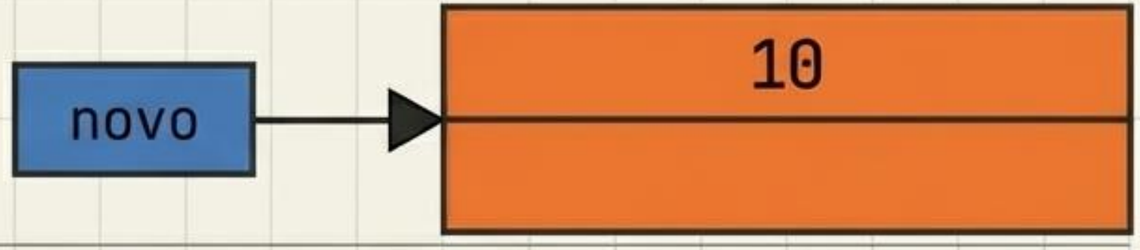
A Linha de Montagem: criar_no(int valor)

Traduzindo código em arquitetura espacial.

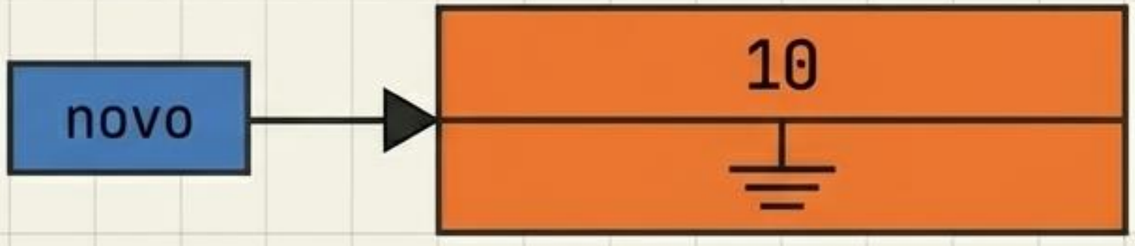
```
No *novo = malloc(sizeof(No));
```

```
if (novo != NULL)
```

```
novo->dado = valor;
```




```
novo->proximo = NULL;
```



NotebookLM

A Patologia: Memory Leak

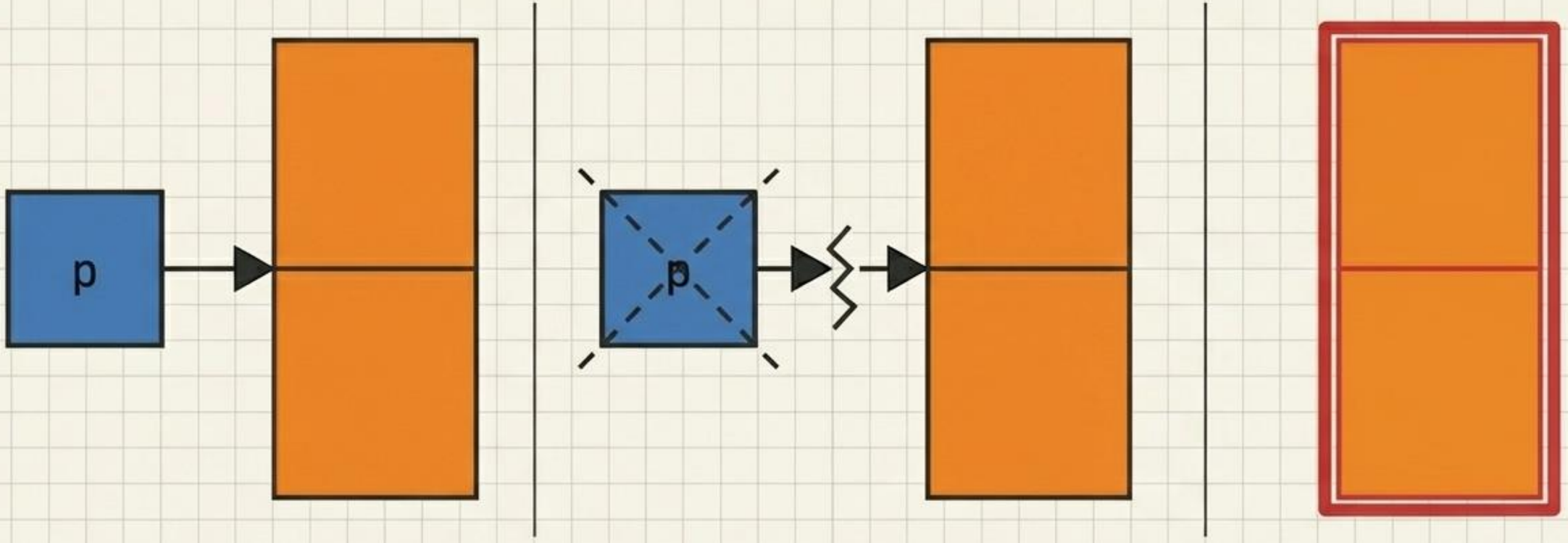
Se perdermos o ponteiro original antes de chamar o free(), a memória fica ocupada para o Sistema Operacional, mas totalmente inacessível para o programa.

```
void erro_memoria() {
   No *p = criar_no(10);

   // Fim da funcao: ‘p’ e destruido na Stack.
   // O espaco no Heap continua ocupado.
   // Correcao: free(p); antes de fechar a chave.
}
```

NotebookLM

Anatomia de um Vazamento

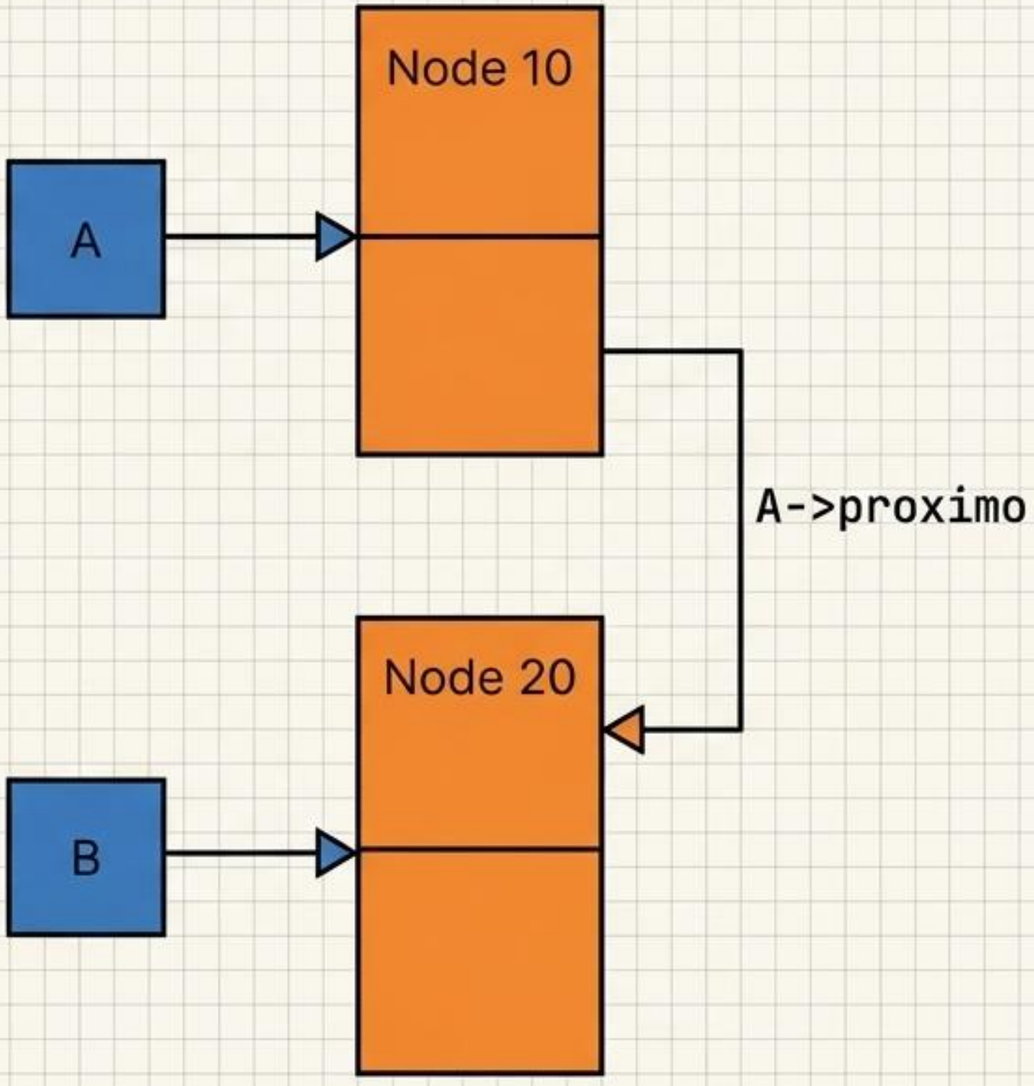


1. Conectado
2. Fim da Função
3. Espaço Perdido (Zombie Memory)

NotebookLM

Atividade Prática: Construindo Conexões

```
No *A = criar_no(10);
No *B = criar_no(20);
A->proximo = B;
```



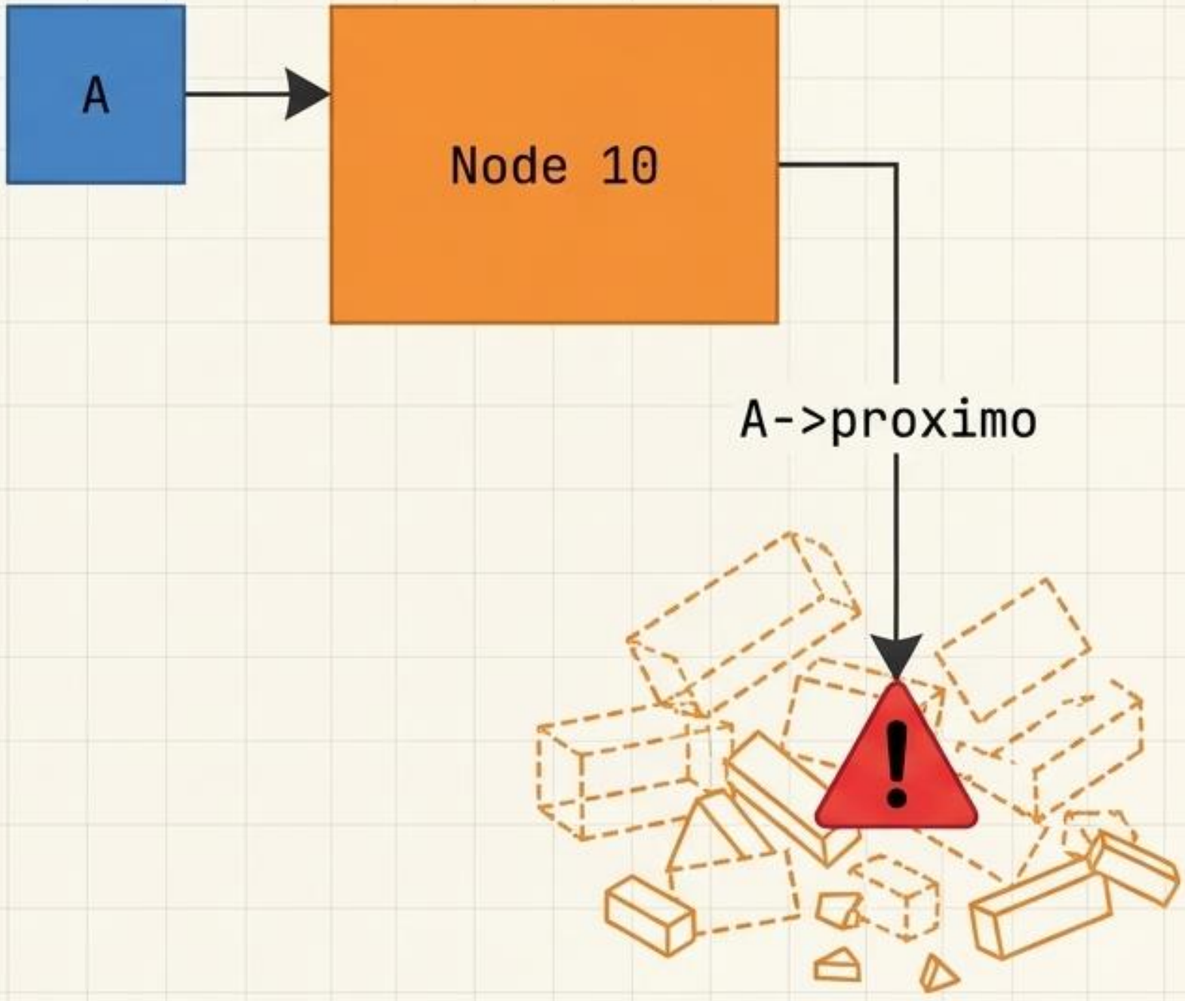
NotebookLM

O Perigo Oculto: O Dangling Pointer

```
free(B);
```

O que acontece com **A->proximo** agora?

A memória foi devolvida ao SO, mas o ponteiro em A não sabe disso. Ele ainda aponta para uma ruína na memória. Tentar acessar A->proximo agora resultará em falha catastrófica. O mapa sobreviveu ao território.



NotebookLM